

**SANJIVANI COLLEGE OF ENGINEERING,  
KOPARGAON**



**(An autonomous Institute affiliated to SPPU Pune)**

**Department of Computer Engineering**

**Academic Year: 2026-27 Sem-I**

**Advance Data Science Lab (PECO412B)**

**Mini Project Activity**

**Group ID: CMG2**

**Title: Power Consumption Prediction**

**Submitted By**

**Mr. Bankar Smitraj Dinkar**

**UCS23M1009**

**Ms. Tanishka Sharad Bhakare**

**UCS23F1011**

**Ms. Shraddha Sunil Anap**

**UCS23F1003**

**Ms. Borse Vaishnavi Sandesh**

**UCS23F1019**

**Course Incharge  
Mr. S. A. Shivarkar**

|                                                                                   |                                                                                                                                                          |                                           |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
|  | <b>Sanjivani Rural Educational Society's<br/>SANJIVANI COLLEGE OF ENGINEERING</b><br>(An Autonomous Institution)<br>Kopargaon – 423 603,<br>Maharashtra. | <b>ACAD-F-15 K</b>                        |
| <b>Academic Year:<br/>2026-27</b>                                                 | <b>Mini Project Activity</b>                                                                                                                             | <b>Revision: 00<br/>Dated :</b>           |
| <b>Department :<br/>Computer</b>                                                  | <b>Computer Engineering</b>                                                                                                                              | <b>Date of Preparation :<br/>08/10/26</b> |
| <b>Course Code &amp;<br/>Name:</b>                                                | <b>PECO412B: Advance Data Science Laboratory</b>                                                                                                         | <b>Year/Sem:<br/>BTech-7th semester</b>   |

**Project Title: Power Consumption Prediction**

**Submitted by:**

| <b>Roll No.</b> | <b>PRN No.</b>    | <b>Name of the Student</b>     | <b>Signature</b> |
|-----------------|-------------------|--------------------------------|------------------|
| <b>03</b>       | <b>UCS23F1003</b> | <b>Anap Shraddha Sunil</b>     |                  |
| <b>09</b>       | <b>UCS23M1009</b> | <b>Bankar Smitraj Dinkar</b>   |                  |
| <b>11</b>       | <b>UCS23F1011</b> | <b>Bhakare Tanishka Sharad</b> |                  |
| <b>19</b>       | <b>UCS23F1019</b> | <b>Borse Vaishnavi Sandesh</b> |                  |

**Course Incharge  
Prof. S. A. Shivarkar**

## ABSTRACT

Reliable electricity demand forecasting is essential for modern smart grids. Over-estimating demand leaves costly reserve plants idling, while under-estimating it risks transformer overloading, voltage degradation and blackouts. This project develops a Smart Grid Power Consumption Forecasting System using deep learning. The dataset contains 52,416 telemetry records sampled every 10 minutes during 2017 from three distribution zones of Tetouan City, Morocco, together with weather measurements (temperature, humidity, wind speed and two solar-diffuse indices).

Data pre-processing includes date parsing, chronological 70/15/15 splitting, engineering of a 17-dimensional feature vector (cyclical calendar harmonics, heat-index proxy, total solar flux, weekend flag and lagged zone loads), standardisation, and a 12-step (2-hour) sliding window. Five deep networks are trained and compared on an unseen chronological test set: Residual Deep MLP, LSTM, GRU, Temporal Transformer and CNN-BiLSTM. The Residual-MLP performs best, with  $R^2 = 0.9964$ , RMSE = 853.09 KW, MAE = 640.93 KW and MAPE = 1.06% on total grid load, at roughly 0.004 ms per sample on CPU. The trained models are served through a FastAPI backend and an interactive web dashboard offering live telemetry streaming, multi-horizon forecasts and a climate what-if simulator.

With the growth of urban demand, climate variability and distributed loads, grid operators need fast and accurate short-term forecasts for each distribution zone. The proposed system analyses weather, calendar and recent load history using neural networks, compares several architectures under a leak-free evaluation protocol, and exposes the best models as a REST service. This helps utilities improve dispatch planning, reduce peak-generation cost and maintain grid stability.

# INDEX

| Chapter No. | Title                                | Page no. |
|-------------|--------------------------------------|----------|
| 1           | Problem Definition.....              | 1        |
| 2           | Introduction.....                    | 3        |
| 3           | System Architecture and working..... | 5        |
| 4           | Code.....                            | 13       |
| 5           | Result Analysis.....                 | 30       |
| 6           | Conclusion.....                      | 34       |
| 7           | References.....                      | 35       |

# CHAPTER 1

## Problem Definition

### Title

Multi-Zone Power Consumption Forecasting for a Smart Grid Using Deep Learning (Case Study: Tetouan City, Morocco)

### Problem Definition

Electricity grids must balance supply and demand continuously. Rapid urban expansion, climate variability and the growth of decentralised loads such as air-conditioning make the demand of a city highly dynamic and strongly dependent on weather and time of day. Grid dispatchers therefore need accurate short-term forecasts for every part of the distribution network, not only for the city total.

Forecasting errors are costly in both directions. Over-estimating demand leads to idling of expensive reserve peaking plants, whereas under-estimating it risks transmission-line congestion, transformer tripping, voltage degradation and brown-outs. Manual estimation and simple statistical methods such as ARIMA struggle with the non-linear relationship between load, temperature, humidity and solar radiation, with multiple daily and seasonal cycles, and with the very different load behaviour of different zones of the same city.

The data used here come from three distribution zones of Tetouan, each with a different consumption profile. A single aggregate model hides these differences, so the problem is to predict the consumption of all three zones simultaneously and also the total grid load, from recent load history, weather and calendar information.

### Objective

The main objective of this project is to develop an intelligent deep-learning-based system that forecasts the next-step power consumption of each of the three zones, and of the whole grid, and makes the forecasts available to users in real time. The specific objectives are:

- To analyse the 52,416-record Tetouan telemetry dataset and understand daily, monthly and weather-driven consumption patterns.
- To engineer informative features (cyclical time encodings, heat-index proxy, solar flux, weekend flag, lagged zone loads) and build a leak-free chronological train / validation / test pipeline.
- To design, train and compare five deep-learning architectures: Residual Deep MLP, LSTM, GRU, Temporal Transformer and CNN-BiLSTM.
- To evaluate every model with  $R^2$ , RMSE, MAE and MAPE for each zone and for the total load, together with inference latency and training time.
- To deploy the trained models as a FastAPI web service with an interactive dashboard for live telemetry, multi-horizon forecasting and climate what-if analysis.

**Scope**

- Automates short-term load forecasting for several distribution zones, reducing manual effort for dispatchers
- Helps avoid transformer overloading, congestion and unnecessary reserve generation by anticipating peaks
- Can be integrated into SCADA / EMS dispatch loops because inference is extremely fast on ordinary CPU hardware
- Uses weather, calendar and load-history data for accurate prediction
- Provides a climate-sensitivity sandbox to study how temperature, humidity and solar changes affect demand
- Scalable to additional zones, longer histories and other cities with the same pipeline

## CHAPTER 2

### Introduction

Electric power is the backbone of modern urban life, and the amount of power a city draws changes every minute with human activity and weather. Utilities must generate and route exactly that amount of power at every instant. As cities grow and as heating, cooling and industrial loads become more variable, load forecasting has become one of the most important functions of a smart grid.

Consumption in a city is not uniform. A downtown commercial area follows office hours, an industrial corridor follows factory shifts, and a residential or tourist area follows household behaviour and the seasons. Weather adds a further layer: temperature, humidity, wind and solar radiation all change how much cooling, heating and lighting is required. These interacting effects make the load series non-linear and non-stationary, which is difficult for manual estimation and for classical linear models.

This project presents a deep-learning-based Power Consumption Forecasting system. Using 10-minute telemetry from three distribution zones of Tetouan City (Morocco) for the whole of 2017, it predicts the next-step load of each zone and of the total grid from the previous two hours of weather, calendar and load data. Five neural architectures (Residual Deep MLP, LSTM, GRU, Temporal Transformer and CNN-BiLSTM) are trained under an identical, strictly chronological protocol so that they can be compared fairly.

The proposed system focuses on accuracy ( $R^2$ , RMSE, MAE, MAPE) and also on speed, because forecasts used in grid operation must be available within milliseconds. The best model is exposed through a FastAPI service and a web dashboard that streams live telemetry, projects load for 6 to 48 hours, and lets operators test climate stress scenarios.

#### 2.1 Related Work

Traditional load forecasting relies on statistical time-series models such as ARIMA and SARIMAX, which model a series as a linear function of its own past and of external regressors. They are interpretable and work well when the series is close to linear, but they have difficulty with strongly non-linear weather effects and with several overlapping seasonal cycles. Machine-learning regressors (for example support-vector regression, random forests and shallow neural networks) improved on them; the Tetouan dataset itself was introduced in a study by Salam and El Hibaoui that compared machine-learning algorithms for power-consumption prediction [2].

Deep learning extends this line of work. Recurrent networks such as LSTM [3] and GRU [4] keep a hidden state across the input window and mitigate vanishing gradients through gating. Convolutional layers combined with bidirectional recurrent layers extract local patterns before sequence modelling. Transformers [5] use self-attention to relate every time step to every other one in parallel. Residual connections [6] allow deeper dense networks to train stably. This project compares one representative of each family on the same data.

## 2.2 Key Contributions

- **Full-spectrum neural comparison:** five deep-learning architectures benchmarked on an identical chronological test set.
- **Physically motivated features:** heat-index proxy and total solar flux combined with sine/cosine calendar encodings.
- **Fast inference:** sub-millisecond per-sample CPU inference for all models, the fastest being the Residual-MLP.
- **Full-stack system:** FastAPI back end with live telemetry streaming, multi-horizon forecasting and a climate what-if sandbox, working without any external cloud service.

## CHAPTER 3

### System Architecture and Working

#### 3.1 Dataset Description (Brief Introduction)

The dataset used in this project is the "Power Consumption of Tetouan City" dataset from the UCI Machine Learning Repository, donated by A. Salam and A. El Hibaoui:

<https://archive.ics.uci.edu/dataset/849/power+consumption+of+tetouan+city>

It contains continuous telemetry from the electricity distribution network of Tetouan, a city in northern Morocco. Each record gives the weather conditions and the power consumed by each of three distribution zones during a 10-minute interval. The file used in the project is power consumption.csv.

#### Key Characteristics

- **Type:** multivariate time-series (tabular) data
- **Size:** 52,416 records  $\times$  9 columns ( $6 \times 24 \times 364$  ten-minute steps)
- **Period:** 1 January 2017 00:00:00 to 30 December 2017 23:50:00, sampled every 10 minutes
- **Targets:** power consumption of Zone 1, Zone 2 and Zone 3 (multi-output regression); Total Power = Zone 1 + Zone 2 + Zone 3
- **Missing values:** none (checked in the EDA script)
- **Availability:** publicly available from the UCI Machine Learning Repository

#### Important Features

The dataset contains a timestamp, five weather attributes and three zone-wise consumption values:

**Table 3.1: Dataset attributes**

| Field                 | Description                                       | Unit / Type                   |
|-----------------------|---------------------------------------------------|-------------------------------|
| DateTime              | 10-minute timestamp of the record                 | Date-time                     |
| Temperature           | Ambient air temperature of Tetouan                | °C (continuous)               |
| Humidity              | Relative humidity                                 | % (continuous)                |
| Wind Speed            | Wind speed                                        | m/s (continuous)              |
| general diffuse flows | General diffuse solar radiation index             | W/m <sup>2</sup> (continuous) |
| diffuse flows         | Diffuse solar radiation index                     | W/m <sup>2</sup> (continuous) |
| Zone 1                | Power consumption of distribution zone 1 (target) | KW (continuous)               |
| Zone 2                | Power consumption of distribution zone 2 (target) | KW (continuous)               |
| Zone 3                | Power consumption of distribution zone 3 (target) | KW (continuous)               |

## Zone Consumption Statistics

**Table 3.2: Zone-wise power consumption statistics (computed from power consumption.csv)**

| Zone              | Mean (KW)        | Std Dev (KW)     | Min (KW)         | Peak (KW)         | Grid Share  |
|-------------------|------------------|------------------|------------------|-------------------|-------------|
| Zone 1            | 32,344.97        | 7,130.56         | 13,895.70        | 52,204.40         | 45.4%       |
| Zone 2            | 21,042.51        | 5,201.47         | 8,560.08         | 37,408.86         | 29.5%       |
| Zone 3            | 17,835.41        | 6,622.17         | 5,935.17         | 47,598.33         | 25.0%       |
| <b>Total Grid</b> | <b>71,222.89</b> | <b>17,143.14</b> | <b>36,785.04</b> | <b>134,208.15</b> | <b>100%</b> |

### Zone Topology (as characterised in the project documentation)

- **Zone 1 – Downtown commercial and administrative area:** largest consumer (45.4% of grid load), driven by retail, offices, lighting and commercial air-conditioning, with predictable business hours.
- **Zone 2 – Industrial parks and manufacturing corridor:** about 29.5% of load, with steady shift-based machinery demand and moderate weekend drops.
- **Zone 3 – Residential suburbs and coastal tourism belt:** about 25.0% of load but the most variable zone, driven by household appliances, decentralised cooling and seasonal tourist influx.

*The zone descriptions above come from the project documentation; the dataset itself labels the zones only as Zone 1, Zone 2 and Zone 3.*

## 3.2 Data Pre-processing Techniques

### 1. Data Loading and Cleaning

Raw telemetry must be organised before modelling. The following steps are applied (scripts/eda.py and scripts/train\_models.py):

- Strip stray spaces from column names
- Parse the DateTime column into a proper date-time type (mixed format) and sort records chronologically
- Verify that there are no missing or null values (none were found)
- Compute the aggregate target Total Power = Zone 1 + Zone 2 + Zone 3

### 2. Exploratory Data Analysis (EDA)

EDA is performed to understand the data before modelling:

- Checking dataset shape, columns, date range and missing values
- Computing descriptive statistics of weather variables and zone loads (Tables 3.2 and 3.3)
- Computing the Pearson correlation matrix between weather variables and loads (Figure 3.2)
- Computing average hourly and monthly load profiles (Figure 3.1)

Key observations: the total load is lowest around 06:00 (average 50,190 KW) and peaks in the evening around 20:00 (average 98,037 KW). Monthly averages are highest in July (88,174 KW) and August (85,740 KW) and lowest in December (63,751 KW), reflecting summer cooling

demand. Temperature has the strongest positive correlation with total load ( $r = +0.488$ ) and humidity a moderate negative one ( $r = -0.299$ ).

**Table 3.3: Meteorological covariates and their correlation with total power**

| Weather Variable      | Unit             | Mean   | Min   | Max      | Std Dev | r with Total Power |
|-----------------------|------------------|--------|-------|----------|---------|--------------------|
| Temperature           | °C               | 18.81  | 3.25  | 40.01    | 5.82    | +0.488             |
| Relative Humidity     | %                | 68.26  | 11.34 | 94.80    | 15.55   | -0.299             |
| Wind Speed            | m/s              | 1.96   | 0.05  | 6.48     | 2.35    | +0.222             |
| General Diffuse Flows | W/m <sup>2</sup> | 182.70 | 0.00  | 1,163.00 | 264.40  | +0.150             |
| Diffuse Flows         | W/m <sup>2</sup> | 75.03  | 0.01  | 936.00   | 124.21  | +0.032             |

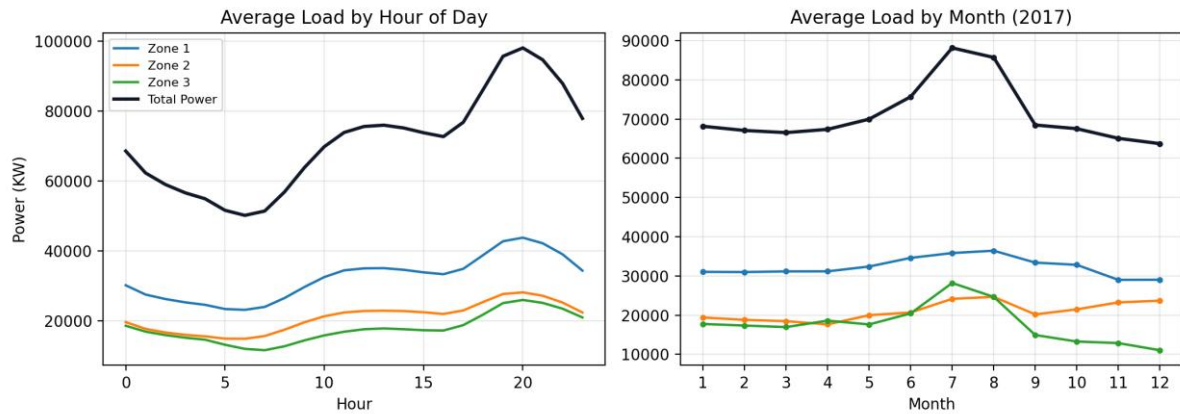


Figure 3.1: Average load by hour of day and by month (from eda\_summary.json)

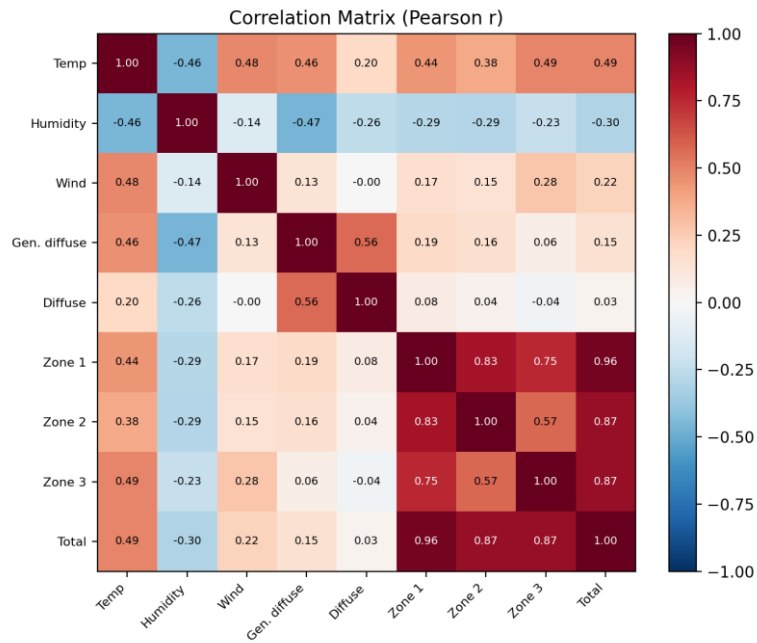


Figure 3.2: Pearson correlation matrix of weather variables and zone loads

### 3. Chronological Data Splitting

For time series, random shuffling would leak future information into training. The records are therefore split strictly in time order:

**Table 3.4: Chronological split of the dataset**

| Partition       | Share | Records               | Period                                |
|-----------------|-------|-----------------------|---------------------------------------|
| Training        | 70%   | 36,691                | 1 Jan 2017 to about 13 Sep 2017       |
| Validation      | 15%   | 7,862                 | about 13 Sep 2017 to about 6 Nov 2017 |
| Test (hold-out) | 15%   | 7,863 (7,851 windows) | about 6 Nov 2017 to 30 Dec 2017       |

#### 4. Feature Preparation (17-Dimensional Feature Vector)

Each time step is projected into a 17-dimensional feature vector made of the following groups:

**Table 3.5: Composition of the feature vector**

| Group                | Features                                                                | Dims |
|----------------------|-------------------------------------------------------------------------|------|
| Weather telemetry    | Temperature, Humidity, Wind Speed, general diffuse flows, diffuse flows | 5    |
| Physical proxies     | HeatIndexProxy, TotalSolarFlux                                          | 2    |
| Cyclical encodings   | sin/cos of hour (24), month (12) and day of week (7)                    | 6    |
| Calendar flag        | IsWeekend (Saturday/Sunday = 1)                                         | 1    |
| Autoregressive state | Zone 1, Zone 2 and Zone 3 load at the current step                      | 3    |

$$\text{HeatIndexProxy} = (\text{Temperature} \times \text{Humidity}) / 100$$

$$\text{TotalSolarFlux} = \text{general diffuse flows} + \text{diffuse flows}$$

$$\sin(2\pi \cdot \text{Hour}/24), \cos(2\pi \cdot \text{Hour}/24), \sin(2\pi \cdot \text{Month}/12), \cos(2\pi \cdot \text{Month}/12), \sin(2\pi \cdot \text{DoW}/7), \cos(2\pi \cdot \text{DoW}/7)$$

#### Engineered Features and Rationale

**Table 3.6: Engineered features**

| Feature             | Rationale                                                                        |
|---------------------|----------------------------------------------------------------------------------|
| HeatIndexProxy      | Combines temperature and humidity; hot and humid conditions raise cooling demand |
| TotalSolarFlux      | Total diffuse solar radiation; relates to daylight, lighting and cooling load    |
| sin/cos Hour        | Encodes the daily cycle continuously so that 23:50 and 00:00 are close           |
| sin/cos Month       | Encodes the seasonal cycle (summer cooling, winter heating)                      |
| sin/cos Day of week | Captures weekly routine of offices, factories and households                     |
| IsWeekend           | Commercial and industrial demand drops on weekends                               |
| Zone 1/2/3 (lagged) | Recent load is the strongest predictor of the next 10-minute load                |

#### 5. Feature Scaling

Inputs and targets are standardised with StandardScaler. The scalers are fitted only on the training partition and then applied to validation and test data, so no information from the future leaks into training. The mean and scale values are saved in models/scalers.json for use by the API.

$$z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$$

#### 6. Sliding-Window Construction

- **Lookback window W:** 12 consecutive 10-minute steps (2 hours)
- **Model input:** tensor X of shape  $B \times 12 \times 17$  ( $B$  = batch size)

- **Target:** Y of shape  $B \times 3$ , the Zone 1, Zone 2 and Zone 3 load at the next step ( $t + 1$ )
- **Stride:** 2 for training and validation windows (18,340 and 3,925 windows) and 1 for the test set (7,851 windows)

## 7. Handling Outliers and Sensor Noise

Instead of resampling (class imbalance does not arise in regression), robustness to occasional sensor spikes is obtained by using the Huber loss ( $\delta = 1.0$ ) together with gradient clipping (max-norm 2.0) during training.

## 3.3 System Architecture

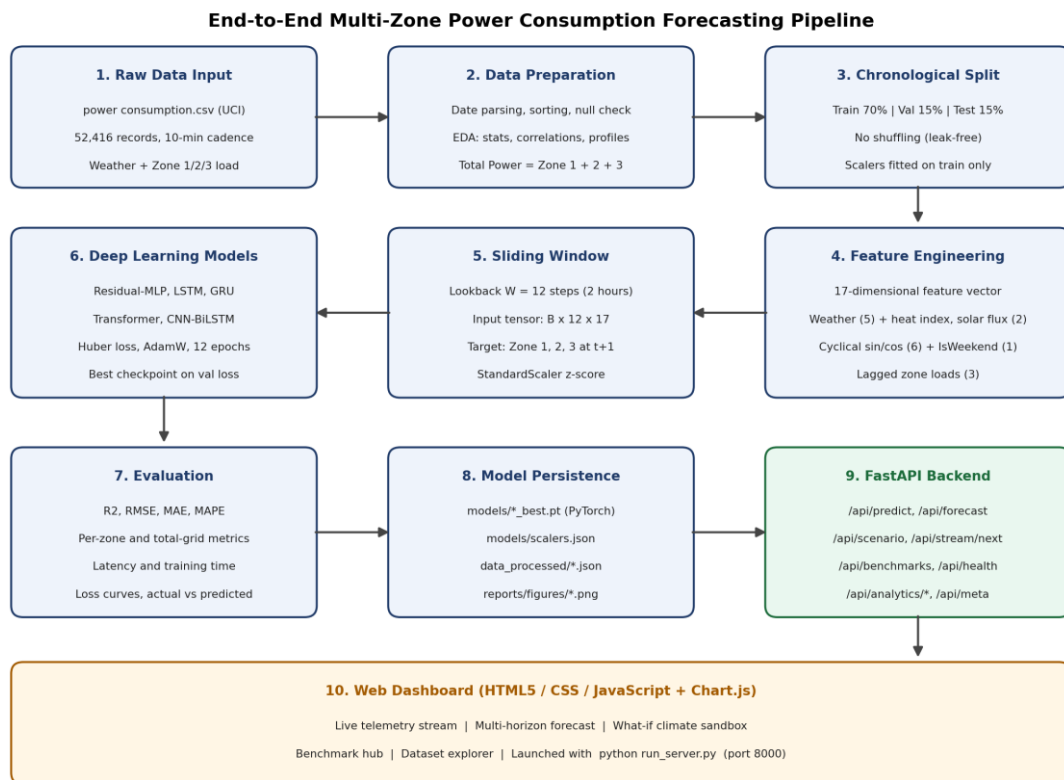


Figure 3.3: System Architecture Diagram

- **Input Layer:** raw telemetry (weather + zone loads) from power consumption.csv
- **Preprocessing Layer:** cleaning, EDA and chronological splitting
- **Feature Engineering Layer:**
  - Cyclical calendar encodings and weekend flag
  - Heat-index proxy and total solar flux
  - Lagged zone loads, standardisation and 12-step sliding window
- **Model Layer (five deep networks):**
  - Residual Deep MLP (dense ResNet)
  - LSTM and GRU (recurrent)
  - Temporal Transformer (self-attention)
  - CNN + Bidirectional LSTM (hybrid)

- **Evaluation Layer:**  $R^2$ , RMSE, MAE, MAPE per zone and total; inference latency; training time
- **Deployment Layer:** FastAPI back end + HTML5/JavaScript dashboard (Chart.js) served on port 8000

### 3.3.1 Model Architectures

All models are defined in backend/models\_definition.py and share the same input ( $B \times 12 \times 17$ ) and output ( $B \times 3$ ):

**Table 3.7: Summary of the five architectures**

| Model        | Configuration                                                                                                                                                               | Parameters | Latency (ms/sample) |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------------|
| Residual-MLP | Flatten 204 $\rightarrow$ Linear 128; two residual blocks (LayerNorm $\rightarrow$ Linear $\rightarrow$ GELU $\rightarrow$ Dropout 0.15) with skip connections; linear head | 60,163     | 0.004               |
| LSTM         | 2 layers, hidden 64, dropout 0.2; last-step output $\rightarrow$ 32 $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ 3                                                | 56,707     | 0.065               |
| GRU          | 2 layers, hidden 64, dropout 0.2; last-step output $\rightarrow$ 32 $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ 3                                                | 43,075     | 0.065               |
| Transformer  | Linear projection to d_model 64 + sinusoidal positional encoding; 2 encoder layers, 4 heads, feed-forward 128; mean pooling; 64 $\rightarrow$ 32 $\rightarrow$ 3            | 70,275     | 0.025               |
| CNN-BiLSTM   | Conv1D (k = 3, 64 filters) + BatchNorm + ReLU; 1-layer bidirectional LSTM (hidden 32); 64 $\rightarrow$ 32 $\rightarrow$ 3                                                  | 30,723     | 0.046               |

#### Residual Deep MLP (best-performing model)

The  $12 \times 17$  window is unrolled into a 204-element vector, projected to 128 units and passed through two residual blocks with Layer Normalisation and GELU activation:

$$\begin{aligned}
 h_0 &= W_{in} \cdot \text{vec}(X) + b_{in} \\
 h_1 &= h_0 + \text{Dropout}( \text{GELU}( W_1 \cdot \text{LayerNorm}(h_0) + b_1 ) ) \\
 h_2 &= h_1 + \text{Dropout}( \text{GELU}( W_2 \cdot \text{LayerNorm}(h_1) + b_2 ) ) \\
 \hat{Y} &= W_{out} \cdot h_2 + b_{out}
 \end{aligned}$$

#### LSTM and GRU

The LSTM keeps a cell state and a hidden state controlled by forget, input and output gates, which mitigates vanishing gradients over the window:

$$\begin{aligned}
 f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
 \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t)
 \end{aligned}$$

The GRU simplifies this with an update gate and a reset gate and no separate memory cell, which gives fewer parameters:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

#### Temporal Transformer

Each time step is embedded, given a positional encoding and processed by multi-head self-attention, so that every step can attend to every other step in parallel:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

### CNN-BiLSTM

A one-dimensional convolution (kernel size 3, 64 filters) with batch normalisation extracts local patterns across neighbouring time steps; a bidirectional LSTM then models forward and backward context over the convolved sequence.

### 3.3.2 Training Configuration

**Table 3.8: Training configuration (from scripts/train\_models.py)**

| Setting                  | Value                                                                             |
|--------------------------|-----------------------------------------------------------------------------------|
| Loss function            | Huber loss ( $\delta = 1.0$ ) on standardised targets                             |
| Optimiser                | AdamW, learning rate 1e-3, weight decay 1e-4                                      |
| Learning-rate schedule   | ReduceLROnPlateau (factor 0.5, patience 2) on validation loss                     |
| Epochs / batch size      | 12 epochs; batch size 128 (training), 256 (validation, test)                      |
| Regularisation           | Dropout (0.1 to 0.2) and gradient clipping (max-norm 2.0)                         |
| Model selection          | Checkpoint with the lowest validation loss is saved and evaluated on the test set |
| Reproducibility / device | Random seed 42; training and inference on CPU                                     |

### 3.3.3 Deployment Layer: REST API and Dashboard

The back end (backend/main.py) is a FastAPI application that loads the five checkpoints and the scalars at start-up. Interactive Swagger documentation is available at <http://localhost:8000/docs>.

**Table 3.9: REST API endpoints**

| Endpoint                       | Method | Description                                                                         |
|--------------------------------|--------|-------------------------------------------------------------------------------------|
| /api/health                    | GET    | Health status, device, loaded models, number of records                             |
| /api/meta                      | GET    | Zone metadata, feature list, date range and available models                        |
| /api/benchmarks                | GET    | Test metrics and loss histories of all models                                       |
| /api/predict                   | POST   | Next-step zone and total load for given weather/time inputs, with load-status label |
| /api/forecast                  | POST   | Multi-step forecast over a horizon of 1 to 72 hours using the test-set window       |
| /api/scenario                  | POST   | Climate what-if simulation: temperature, humidity and solar deltas vs. a baseline   |
| /api/stream/next               | GET    | Next packet of the live telemetry stream (actual vs. predicted, alert flag)         |
| /api/power/initial-trajectory  | GET    | Pre-computed actual-vs-predicted trajectory for the dashboard                       |
| /api/power/dataset-records     | GET    | Paginated records of the raw dataset (limit, offset)                                |
| /api/analytics/hourly-patterns | GET    | Hourly and monthly average load profiles                                            |
| /api/analytics/correlations    | GET    | Correlation matrix from the EDA summary                                             |

The web dashboard (frontend\_dist/) is a single-page HTML5/CSS/JavaScript application using a locally bundled Chart.js. Its tabs are: Live Monitor (real-time telemetry stream with Total/Zone 1/2/3 toggles), Forecast (6, 12, 24 and 48-hour projections), Power Estimator (what-if calculator for weather inputs), Dataset (browser for the 52,416 raw records), Benchmarks (model comparison charts), Topology (zone view) and Report (technical summary).

### 3.4 Working

1. **Data Input:** the 10-minute telemetry (weather and Zone 1/2/3 consumption) is loaded from power consumption.csv.
2. **Data pre-processing:** column names are cleaned, timestamps parsed and sorted, null values checked, and Total Power computed.
3. **Exploratory Data Analysis (EDA):** statistics, correlations and hourly/monthly profiles are computed and saved to data\_processed/eda\_summary.json.
4. **Data splitting:** the series is split chronologically into 70% training, 15% validation and 15% test data, with no shuffling.
5. **Feature Engineering:** the 17 features (weather, heat index, solar flux, cyclical encodings, weekend flag, lagged zone loads) are generated and standardised with scalers fitted on the training set only.
6. **Windowing:** sliding windows of 12 steps (2 hours) are built; the target is the next-step load of the three zones.
7. **Model Training:** Residual-MLP, LSTM, GRU, Transformer and CNN-BiLSTM are trained with Huber loss and AdamW for 12 epochs; the best checkpoint of each model (lowest validation loss) is saved as a .pt file.
8. **Model Evaluation:** predictions on the unseen test windows are converted back to KW and scored with  $R^2$ , RMSE, MAE and MAPE per zone and for the total grid; latency and training time are recorded in model\_benchmarks.json.
9. **Deployment and Serving:** FastAPI loads the checkpoints and scalers, exposes the REST endpoints and serves the dashboard.
10. **Visualisation and Decision Support:** the dashboard shows live actual-vs-predicted load, multi-hour projections and climate scenarios, with peak-load and anomaly alerts for operators.

# CHAPTER 4

## Code

### 4.1 File Structure

```
MID_TERM_ADSL/
-- backend/
-- main.py          # FastAPI application and API routing
-- models_definition.py # PyTorch implementations of the 5 models
-- data_processed/
-- eda_summary.json  # Statistics, correlations, hourly/monthly profiles
-- model_benchmarks.json # Test metrics and training-loss histories
-- sample_stream.csv  # Telemetry cache (first 2,000 rows)
-- * sample_preds.json # Test predictions per model (48 hours)
-- frontend_dist/
-- index.html        # Single-page dashboard
-- static/
-- css/style.css      # Dashboard styling
-- js/
-- app.js            # Telemetry streaming and Chart.js logic
-- chart.umd.min.js   # Local copy of Chart.js
-- models/
-- residual_mlp_best.pt # Residual-MLP checkpoint (best model)
-- lstm_best.pt         # LSTM checkpoint
-- gru_best.pt          # GRU checkpoint
-- transformer_best.pt  # Transformer checkpoint
-- cnn_bilstm_best.pt   # CNN-BiLSTM checkpoint
-- scalers.json         # Feature and target scalers
-- reports/
-- figures/             # loss curves, model comparison bar, actual vs predicted
-- PRODUCTION_REPORT.md # Technical report (markdown)
-- scripts/
-- eda.py              # Exploratory data analysis
-- train_models.py     # Training, benchmarking, figure generation
-- power_consumption.csv # Raw telemetry dataset (52,416 rows)
-- run_server.py       # Server launcher
-- README.md           # Project documentation
```

*Note: the compiled pycache folders are omitted.*

### 4.2 Software Requirements

Table 4.1: Software requirements

| Component                  | Purpose                                                           |
|----------------------------|-------------------------------------------------------------------|
| Python 3.10 or higher      | Programming language                                              |
| PyTorch 2.0+               | Neural network definition, training and inference                 |
| pandas, NumPy              | Data loading, feature engineering and numerical computing         |
| scikit-learn               | StandardScaler and evaluation metrics (MAE, MSE, R <sup>2</sup> ) |
| matplotlib, seaborn        | Figures for the report                                            |
| FastAPI, Uvicorn, Pydantic | REST API server and request/response schemas                      |
| Chart.js (bundled locally) | Dashboard charts                                                  |

### 4.3 Execution Guide

#### Quick start (pre-trained models included)

```
# 1. Go to the project root
cd MID_TERM_ADSL

# 2. (First time only) install the dependencies listed in Table 4.1
pip install -r requirements.txt

# 3. Launch the FastAPI server and dashboard
python run_server.py
```

Open <http://localhost:8000> for the dashboard and <http://localhost:8000/docs> for the Swagger API documentation.

#### Complete pipeline from scratch

```
# Step 1: Exploratory Data Analysis -> data_processed/eda_summary.json
```

```
python scripts/eda.py

# Step 2: Train and benchmark all 5 models -> models/*.pt, model_benchmarks.json, figures
python scripts/train_models.py

# Step 3: Launch the production server
python run_server.py
```

## 4.4 run\_server.py

```
import uvicorn
import sys
import os

# Ensure project root is in sys.path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

if __name__ == "__main__":
    print("=" * 65)
    print(" TETOUAN SMART GRID DEEP LEARNING POWER FORECASTING SYSTEM")
    print(" Starting FastAPI Server & Neural Inference Engine...")
    print(" Web Dashboard: http://localhost:8000")
    print(" API Documentation: http://localhost:8000/docs")
    print("=" * 65)
    uvicorn.run("backend.main:app", host="127.0.0.1", port=8000, reload=False)
```

*The front-end files (index.html, static/js/app.js, static/css/style.css) are supplied in the project folder and are not reproduced here.*

## CHAPTER 5

### Result Analysis

#### 5.1 Evaluation Metrics

Predictions on the test windows are inverse-transformed to KW and evaluated with the following metrics ( $y$  = actual,  $\hat{y}$  = predicted,  $N$  = number of test windows):

$$MAE = (1/N) \sum |y_i - \hat{y}_i|$$

$$RMSE = \sqrt{(1/N) \sum (y_i - \hat{y}_i)^2}$$

$$MAPE = (100\% / N) \sum |(y_i - \hat{y}_i) / y_i|$$

$$R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$$

#### 5.2 Model Performance Comparison

All models are evaluated on the same 7,851 unseen chronological test windows (about 6 Nov to 30 Dec 2017). The two baselines were computed from the same test windows for reference.

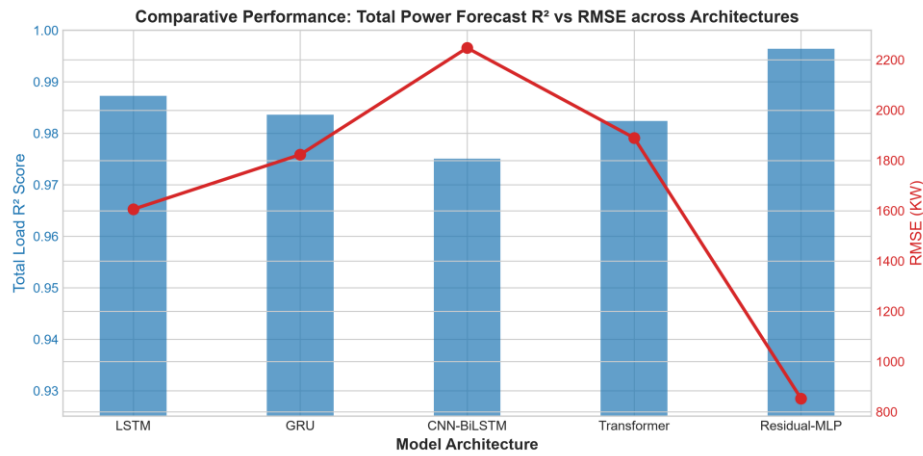
**Table 5.1: Performance comparison on total grid load**

| Model                                             | Params | R <sup>2</sup> | RMSE (KW) | MAE (KW)  | MAPE (%) | Latency (ms/sample) | Train time (s) |
|---------------------------------------------------|--------|----------------|-----------|-----------|----------|---------------------|----------------|
| Baseline 1: train-set mean                        | —      | -0.3908        | 16,805.31 | 14,552.87 | 25.91    | —                   | —              |
| Baseline 2: persistence ( $\hat{y}(t+1) = y(t)$ ) | —      | 0.9932         | 1,177.87  | 786.15    | 1.24     | —                   | —              |
| Residual-MLP                                      | 60,163 | 0.9964         | 853.09    | 640.93    | 1.06     | 0.004               | 14.5           |
| LSTM                                              | 56,707 | 0.9873         | 1,605.74  | 1,301.92  | 2.05     | 0.065               | 47.8           |
| GRU                                               | 43,075 | 0.9836         | 1,823.03  | 1,580.88  | 2.68     | 0.065               | 74.0           |
| Transformer                                       | 70,275 | 0.9824         | 1,889.89  | 1,529.60  | 2.48     | 0.025               | 97.8           |
| CNN-BiLSTM                                        | 30,723 | 0.9751         | 2,247.56  | 1,960.72  | 3.34     | 0.046               | 38.9           |

#### Model Performance Comparison

- The Residual-MLP is the best of the five deep models on every aggregate metric:  $R^2 = 0.9964$ ,  $RMSE = 853.09$  KW,  $MAE = 640.93$  KW and  $MAPE = 1.06\%$ . It also has the lowest latency (0.004 ms/sample) and the shortest training time (14.5 s).
- Compared with the next-best deep model (LSTM), the Residual-MLP lowers the total-load RMSE by about 47% (1,605.74 → 853.09 KW); compared with the weakest (CNN-BiLSTM), by about 62%.
- The ranking by total-load  $R^2$  is Residual-MLP (0.9964) > LSTM (0.9873) > GRU (0.9836) > Transformer (0.9824) > CNN-BiLSTM (0.9751).
- The Transformer has a slightly lower MAPE (2.48%) than the GRU (2.68%) although its  $R^2$  is lower, because MAPE and  $R^2$  weight errors differently.
- Against the baselines, the Residual-MLP is clearly better than the train-mean baseline ( $R^2 = -0.39$ ) and also improves on the persistence baseline in total RMSE (853 vs 1,178 KW), MAE (641 vs 786 KW) and MAPE (1.06% vs 1.24%). The LSTM, GRU, Transformer and CNN-BiLSTM have a higher total RMSE than persistence.

- Latency is the average per-sample inference time measured over test batches of 256 samples on CPU; single-request latency through the API is higher because of request handling.



Graph 5.1: Total-load R<sup>2</sup> (bars) and RMSE (line) of the five architectures

### Comparison of Models (Graph 5.1)

- The bars show R<sup>2</sup> and the red line shows RMSE of total load for each architecture on the test set.
- Residual-MLP has the highest R<sup>2</sup> and the lowest RMSE; CNN-BiLSTM has the lowest R<sup>2</sup> and the highest RMSE.
- LSTM, GRU and Transformer lie in between, with similar R<sup>2</sup> values (0.982 to 0.987).

## 5.3 Zone-wise Analysis

Table 5.2: Zone-wise test metrics of the Residual-MLP compared with the persistence baseline

| Grid Zone         | R <sup>2</sup> | RMSE (KW) | MAE (KW) | MAPE (%) | Persistence RMSE (KW) | Persistence MAPE (%) |
|-------------------|----------------|-----------|----------|----------|-----------------------|----------------------|
| Zone 1            | 0.9945         | 448.30    | 334.97   | 1.25     | 537.79                | 1.27                 |
| Zone 2            | 0.9919         | 504.60    | 367.32   | 1.50     | 466.56                | 1.35                 |
| Zone 3            | 0.9715         | 567.25    | 473.85   | 4.22     | 345.14                | 1.76                 |
| <b>Total Grid</b> | 0.9964         | 853.09    | 640.93   | 1.06     | 1,177.87              | 1.24                 |

Table 5.3: Zone-wise RMSE (KW) and R<sup>2</sup> of all models

| Model        | Z1 RMSE  | Z2 RMSE  | Z3 RMSE  | Z1 R <sup>2</sup> | Z2 R <sup>2</sup> | Z3 R <sup>2</sup> |
|--------------|----------|----------|----------|-------------------|-------------------|-------------------|
| Residual-MLP | 448.30   | 504.60   | 567.25   | 0.9945            | 0.9919            | 0.9715            |
| LSTM         | 985.85   | 1,672.93 | 1,954.25 | 0.9736            | 0.9105            | 0.6618            |
| GRU          | 949.54   | 1,485.09 | 1,999.33 | 0.9755            | 0.9295            | 0.6461            |
| Transformer  | 708.01   | 979.80   | 1,590.98 | 0.9864            | 0.9693            | 0.7759            |
| CNN-BiLSTM   | 1,017.27 | 1,350.41 | 2,509.47 | 0.9718            | 0.9417            | 0.4424            |

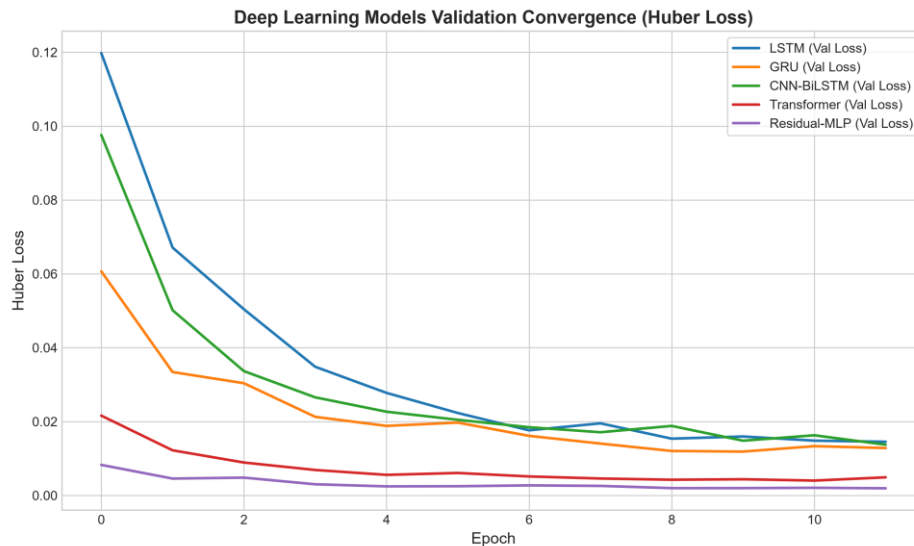
**Table 5.4: Zone-wise MAE (KW) and MAPE (%) of all models**

| Model        | Z1 MAE | Z2 MAE   | Z3 MAE   | Z1 MAPE | Z2 MAPE | Z3 MAPE |
|--------------|--------|----------|----------|---------|---------|---------|
| Residual-MLP | 334.97 | 367.32   | 473.85   | 1.25    | 1.50    | 4.22    |
| LSTM         | 807.61 | 1,459.61 | 1,612.13 | 2.75    | 5.92    | 14.29   |
| GRU          | 801.67 | 1,106.41 | 1,769.46 | 3.08    | 4.20    | 15.71   |
| Transformer  | 584.23 | 566.64   | 1,296.74 | 2.05    | 2.07    | 12.30   |
| CNN-BiLSTM   | 860.42 | 1,007.77 | 2,238.45 | 3.20    | 3.88    | 20.17   |

### Zone-wise Observations

- For the Residual-MLP, Zone 1 ( $R^2 = 0.9945$ , MAPE = 1.25%) and Zone 2 ( $R^2 = 0.9919$ , MAPE = 1.50%) are predicted most accurately; Zone 3 has the largest relative error (MAPE = 4.22%,  $R^2 = 0.9715$ ).
- Zone 3 is also the hardest zone for every other model:  $R^2$  falls to 0.78 (Transformer), 0.66 (LSTM), 0.65 (GRU) and 0.44 (CNN-BiLSTM). The project documentation attributes this to household behaviour, decentralised cooling and tourist influx, which are less regular than commercial or industrial schedules.
- The persistence baseline is strong at 10-minute resolution. It is slightly better than the Residual-MLP on Zone 2 and Zone 3 (for example Zone 3 MAPE 1.76% vs 4.22%), while the Residual-MLP is better on Zone 1 and on the total load. The total-load MAPE (1.06%) is lower than the zone-level MAPE values partly because errors of different zones partially cancel in the sum.

## 5.4 Training Behaviour

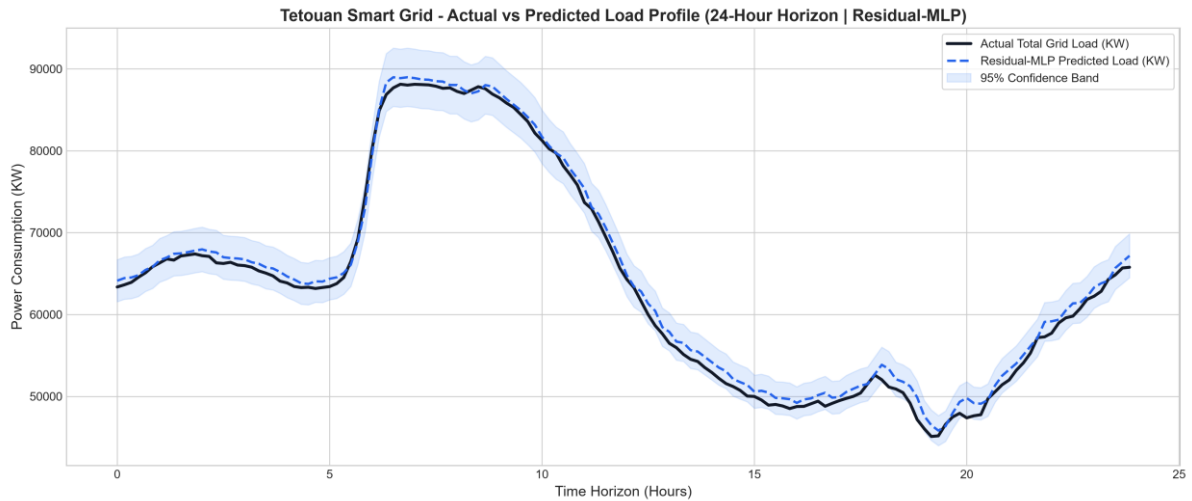


Graph 5.2: Validation Huber loss per epoch for the five models

### Training and Validation Loss (Graph 5.2)

- All models converge smoothly. At epoch 12 the validation loss is 0.00193 for the Residual-MLP, 0.00490 for the Transformer, 0.01283 for the GRU, 0.01373 for the CNN-BiLSTM and 0.01453 for the LSTM.

- The Residual-MLP reaches a low loss within the first few epochs, while the recurrent models start higher and decrease more slowly.
- The recurrent models were still improving slowly at epoch 12, so their results reflect the fixed 12-epoch budget used for a fair comparison; longer training may narrow the gap.



Graph 5.3: Actual vs predicted total grid load over 24 hours (Residual-MLP, test set)

### Actual vs Predicted Load (Graph 5.3)

- The predicted curve follows the actual total load closely over the whole 24-hour window, including the sharp morning rise from about 65,000 to 88,000 KW and the evening trough.
- Small deviations are visible, mainly a slight over-estimation during the afternoon and evening hours.
- The shaded band in this figure is a fixed  $\pm 4\%$  band drawn around the prediction for visualisation; it is not a statistical confidence interval. The API likewise returns a  $\pm 5\%$  interval around its estimate.

## 5.5 Discussion and Insights

- **Why the Residual-MLP performs well:** at 10-minute sampling the load is strongly auto-correlated and smooth, so a dense network that sees the whole 2-hour window at once can combine the latest loads and the weather directly. Residual skip connections and layer normalisation keep training stable. Recurrent networks process the window step by step and, under the same short training budget, did not reach the same accuracy. This is the likely explanation rather than a proven one.
- **Importance of baselines:** a naive persistence forecast already achieves total  $R^2 = 0.9932$  at this horizon, because the input includes the current zone loads. A model should therefore be judged by how much it improves on persistence; the Residual-MLP does so for total load and Zone 1, whereas the other four models do not improve on persistence in total RMSE.
- **Edge deployment:** the Residual-MLP has about 60 thousand parameters and a measured per-sample inference time of 0.004 ms on a CPU. Models of this size can run on low-cost substation hardware without a GPU.

- **Operational use:** the dashboard converts predictions into load-status labels (off-peak trough below 45,000 KW, high-load warning above 80,000 KW, critical peak above 95,000 KW) and scenario recommendations, supporting dispatch decisions.

## CHAPTER 6

### Conclusion

The Smart Grid Power Consumption Forecasting system shows how deep learning can be used to forecast electricity demand in a multi-zone city network. The project processed 52,416 ten-minute records from three zones of Tetouan City, engineered a 17-dimensional feature vector, and trained five neural architectures under a strict chronological, leak-free protocol.

Among the five deep architectures, the Residual Deep MLP gave the best results on the unseen test period, with  $R^2 = 0.9964$ , RMSE = 853.09 KW, MAE = 640.93 KW and MAPE = 1.06% on total grid load, while also being the fastest to train and to run (about 0.004 ms per sample on a CPU). It also improved on a naive persistence baseline for total load and for Zone 1. Zone 3, the most variable zone, remained the hardest to predict for all models.

By combining the trained models with a FastAPI back end and an interactive dashboard, the system gives operators a simple platform for live monitoring, multi-hour projection and climate what-if analysis. The use of machine learning improves efficiency over manual estimation and supports better dispatch planning, lower peak-generation costs and a more stable grid.

#### 6.1 Limitations

- The data cover a single year (2017), so seasonal patterns are seen only once and holiday effects are not modelled.
- The models are trained for next-step (10-minute-ahead) prediction. The dashboard's multi-hour views apply this one-step model to each future step using that step's actual weather, rather than a recursive forecast from predicted loads.
- The `/api/predict` and `/api/scenario` endpoints build their input window from the supplied weather and time values together with fixed typical zone loads, so their outputs are scenario estimates and not forecasts from real recent load history.
- The persistence baseline is competitive in Zone 2 and Zone 3, so the learned models have room to improve there.
- All models were trained for 12 epochs without hyper-parameter search.

#### 6.2 Future Scope

- Edge deployment of the Residual-MLP on substation RTUs/PLCs or single-board computers.
- Predicting the change from the last observed load (residual learning) and training for multiple horizons, to beat persistence in every zone.
- Integration of solar PV generation and storage data, and reinforcement-learning-based dispatch.
- Probabilistic forecasting with calibrated prediction intervals instead of fixed percentage bands.

- Use of multi-year data, holiday and tourism calendars, longer training and hyper-parameter tuning.
- Real-time connection to live SCADA/smart-meter feeds instead of replayed test data.

## CHAPTER 7

### References

- [1] A. Salam and A. El Hibaoui, "Power Consumption of Tetouan City," UCI Machine Learning Repository, 2018.  
<https://archive.ics.uci.edu/dataset/849/power+consumption+of+tetouan+city>
- [2] A. Salam and A. El Hibaoui, "Comparison of Machine Learning Algorithms for the Power Consumption Prediction: Case Study of Tetouan city," 2018 6th International Renewable and Sustainable Energy Conference (IRSEC), pp. 1–5, 2018.  
<https://doi.org/10.1109/IRSEC.2018.8703007>
- [3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," Proc. EMNLP, 2014.
- [5] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [6] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," Proc. IEEE CVPR, pp. 770–778, 2016.
- [7] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," Proc. ICLR, 2019.
- [8] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.
- [9] FastAPI documentation. <https://fastapi.tiangolo.com>
- [10] Chart.js documentation. <https://www.chartjs.org>